Q3 Computational Problem: Synthetic Data

In this problem, we will fit penalized logistic regression models for simple 2D synthetic datasets.

Helper Code

In [2]:
```
import matplotlib.pyplot as plt
from sklearn import datasets
%pip install cvxpy
%pip install ecos
import cvxpy as cp
import numpy as np
from scipy.special import expit as inverse_logit

#### HELPER FUNCTIONS ####
def compute_log_loss(y_true, y_proba, eps=1e-15):
    """
    computes mean logistic loss using labels and probability predictions
    :param y_true: vector of true labels - y_true[i] in (-1,+1) and y_true[i] (0,1) are both OK
    :param y_proba: vector of predicted probabilities
    :param eps: minimum distance that y_pred must maintain from 0 and 1
    :return: log_loss
    """
    p = np.clip(y_proba, eps, 1.0 - eps)
    y = y_true > 0
    L = y * np.log(p) + (1 - y) * np.log(1.0 - p)
    return -np.mean(L)

def compute_auc(y_true, y_proba):
    """
    computes AUC for a binary classifier quickly
    :param y_true: vector of true classes
    :param y_proba: vector of predicted probabilities
    :return: auc
    """
    n = len(y_true)
    I_pos = y_true > 0
    # if y[i] == 0 for all i or y[i] == 1 for all i, then return AUC = 1
    if I_pos.all() or np.logical_not(I_pos).all():
        return float('nan')

    I_pos = I_pos[np.argsort(y_proba)]
    false_positive_count = np.cumsum(1 - I_pos)
    n_false = false_positive_count[-1]
    auc = np.multiply(I_pos, false_positive_count).sum() / (
            n_false * (n - n_false))
    return auc

def compute_ece(y_true, y_proba, n_bins=10):
    """
    computes expected calibration error for a binary classifier
    :param y_true: vector of true classes
    :param y_proba: vector of predicted probabilities
    :param n_bins: 10
    :return:ece
    """

    # pre-process to improve binning
    sort_idx = np.argsort(y_proba)
    y_pred = y_proba[sort_idx]
    y_true = y_true[sort_idx] > 0
    n = len(y_true)

    acc = np.zeros(n_bins)
    conf = np.zeros(n_bins)
    counts = np.zeros(n_bins)

    for k in range(n_bins):
        left_idx = np.searchsorted(y_pred, k / n_bins, side='left')
        right_idx = np.searchsorted(y_pred, (k + 1) / n_bins, side='right')
        if left_idx < right_idx:
            bin_idx = np.arange(left_idx, right_idx)
            counts[k] = len(bin_idx)
            conf[k] = np.mean(y_pred[bin_idx])
            acc[k] = np.mean(y_true[bin_idx])

    ece = np.sum(counts * np.abs(acc - conf)) / n
    return ece

def plot_data(X, y):
    pos_idx = np.greater(y, 0)
    neg_idx = ~pos_idx
```

```
    plt.plot(X[pos_idx, 0], X[pos_idx, 1], 'b.', markersize = 10, alpha = 0.3)
    plt.plot(X[neg_idx, 0], X[neg_idx, 1], 'r.',  markersize = 10,  alpha = 0.3)
    plt.show()
```

```
Requirement already satisfied: cvxpy in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (1.7.3)
Requirement already satisfied: osqp>=1.0.0 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from cvxpy) (1.0.5)
Requirement already satisfied: clarabel>=0.5.0 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from cvxpy) (0.11.1)
Requirement already satisfied: scs>=3.2.4.post1 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from cvxpy) (3.2.9)
Requirement already satisfied: numpy>=1.22.4 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from cvxpy) (2.1.1)
Requirement already satisfied: scipy>=1.13.0 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from cvxpy) (1.14.1)
Requirement already satisfied: cffi in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from clarabel>=0.5.0->cvxpy) (1.
17.1)
Requirement already satisfied: jinja2 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from osqp>=1.0.0->cvxpy) (3.1.
4)
Requirement already satisfied: setuptools in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from osqp>=1.0.0->cvxpy)
(75.1.0)
Requirement already satisfied: joblib in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from osqp>=1.0.0->cvxpy) (1.4.
2)
Requirement already satisfied: pycparser in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from cffi->clarabel>=0.5.0-
>cvxpy) (2.22)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from jinja2->osqp>=1.
0.0->cvxpy) (2.1.5)
Note: you may need to restart the kernel to use updated packages.
Collecting ecos
  Downloading ecos-2.0.14-cp312-cp312-win_amd64.whl.metadata (8.2 kB)
Requirement already satisfied: numpy>=1.6 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from ecos) (2.1.1)
Requirement already satisfied: scipy>=0.9 in c:\users\issac\miniforge3\envs\dsc80\lib\site-packages (from ecos) (1.14.1)
Downloading ecos-2.0.14-cp312-cp312-win_amd64.whl (72 kB)
Installing collected packages: ecos
Successfully installed ecos-2.0.14
Note: you may need to restart the kernel to use updated packages.
```

Part (a)

Implement L_2-penalized logistic regression using `cvxpy`. We have provided you all other than the code for the `fit` method. For this quesiton, you will have to complete the implementation for `fit` starting from line 29. You only need to implement an optimization problem that can be solved to return the correct coefficients. We have already defined the variables and parameters, so all that you need to do is to define the correct objective function and create the optimization problem `cvxpy.` This should only require 2 lines of code.

In [3]:
```
class L2LogisticRegression(object):

    def __init__(self, C = 0.0):
        assert np.greater_equal(C, 0.0)
        self.C = cp.Parameter(nonneg=True)
        self.C.value = C

    # main methods
    def check_data(self, X, y):
        assert X.ndim == 2
        assert X.shape[0] > 0
        assert np.isfinite(X).all()
        assert np.isfinite(y).all()
        assert np.isin(y, (0, 1)).all()
        return True

    def fit(self, X, y):
        """
        :return: coefficients of optimal model
        """
        assert self.check_data(X, y)
        self.n = X.shape[0]
        self.d = X.shape[1]
        self.X = X
        self.y = y
        self.w = cp.Variable(self.d)
        self.t = cp.Variable(1)
        ## YOUR CODE STARTS HERE
        J = cp.sum(cp.multiply(self.y, self.X @ self.w + self.t) - cp.logistic(self.X @ self.w + self.t))
        self.problem = cp.Problem(cp.Maximize(J / self.n - self.C * cp.sum_squares(self.w)))
        ## YOUR CODE ENDS HERE
        self.problem.solve(solver=cp.ECOS)

    #### Classifier Properties ###
    @property
    def has_solution(self):
        return self.w.value is not None

    @property
    def intercept(self):
        if self.has_solution:
            return self.t.value
        else:
            return np.nan

    @property
```

```
    def coefs(self):
        if self.has_solution:
            return self.w.value
        else:
            return np.repeat(np.nan, self.d)

    #### prediction API ####
    def decision_function(self, X):
        """
        returns confidence score
        :param X:
        :return: vector of confidence scores for each row in X
        """
        scores = X.dot(self.coefs) + self.intercept
        return scores

    def predict_proba(self, X):
        """
        predicts probability of positive class
        :param X: feature matrix with d columns features
        :return: vector of predicted probabilities for each row in X
        """
        scores = self.decision_function(X)
        probs = np.zeros_like(scores)
        pos_idx = np.greater_equal(scores, 0)
        neg_idx = np.logical_not(pos_idx)
        probs[pos_idx] = 1.0 / (1.0 + np.exp(-scores[pos_idx]))
        probs[neg_idx] = np.exp(scores[neg_idx]) / (1.0 + np.exp(scores[neg_idx]))
        return probs

    def predict(self, X):
        """
        predicts label for each row in X
        :param X: feature matrix with d columns features
        :return: vector of confidence scores for each row in X
        """
        probs = self.predict_proba(X)
        yhat = np.greater(probs, 0.5)
        return yhat
```

Part (b)

Generate a dataset of $n = 1000$ points by calling the following function in the helper code

`X, y = generate_data_4b(n = 1000)`

i) Plot the positive and negative samples in your dataset using the `plot_data` function in the helper code. Use your plot to explain whether you expect this classification task to be *easy* or *hard*.

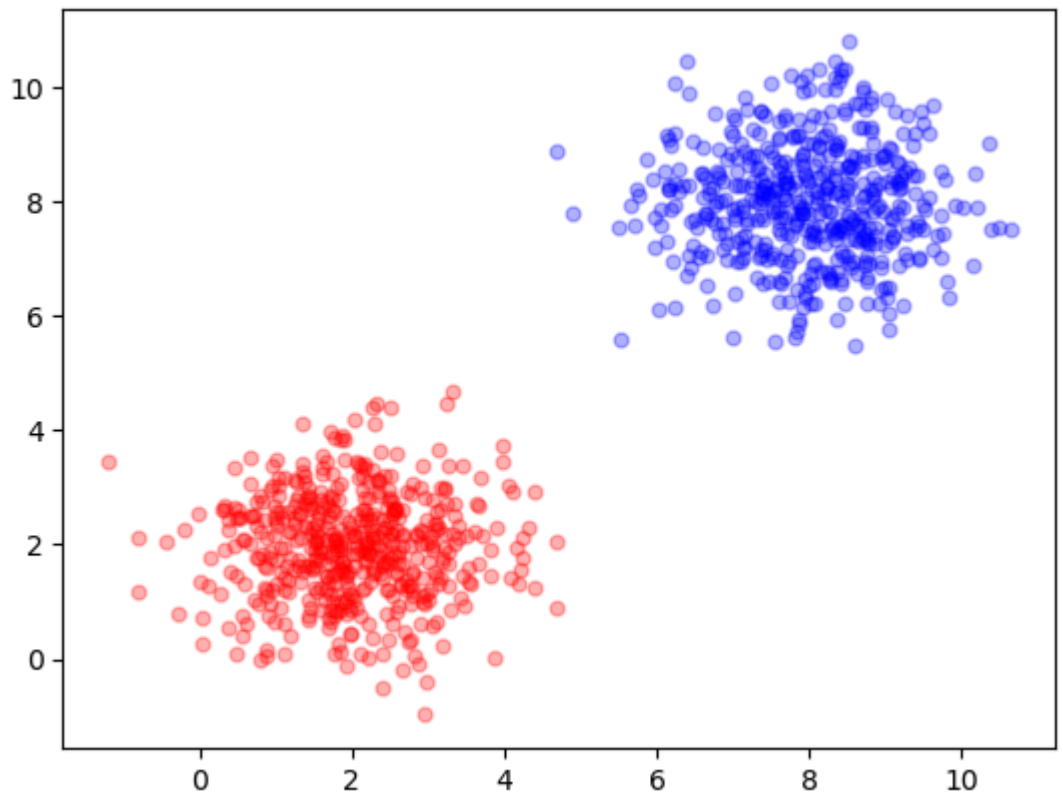
ii) Set the penalty parameter of your model to fit a "vanilla" logistic regression model -- i.e., with no regularization. Report the values of the logistic loss, AUC, and ECE of your model on the training dataset. Write the score function of this model using the fitted values of `L2LogisticRegression.coefs` and `L2LogisticRegression.intercept`

iii) lot a histogram of the probability predictions of the model from on the training dataset. Use these results to explain the behavior of logistic regression on this dataset. Why does `cvxpy` throw a warning in this case?

iv) Fit a penalized logistic regression model on the same dataset where you set the L_2-penalty to $\lambda = 0.1$. How do your results from parts (ii) and (iii) change? Does `cvxpy` still throw a warning?

In [4]:
```
def generate_dataset_4b(n = 1000, seed = 1):
    assert n > 2
    n = int(n)
    separable = False
    while not separable:
        X, y = datasets.make_blobs(n_samples=n, centers=[(2.0, 2.0), (8.0, 8.0)], n_features=2, center_box=(0, 10), random_sta
        red = X[y == 0]
        blue = X[y == 1]
        separable = any([red[:, k].max() < blue[:, k].min() or red[:, k].min() > blue[:, k].max() for k in range(2)])
    return X, y
```

In [5]:
```
## YOUR CODE HERE
X, y = generate_dataset_4b(n=1000, seed=10)
plot_data(X, y)
```

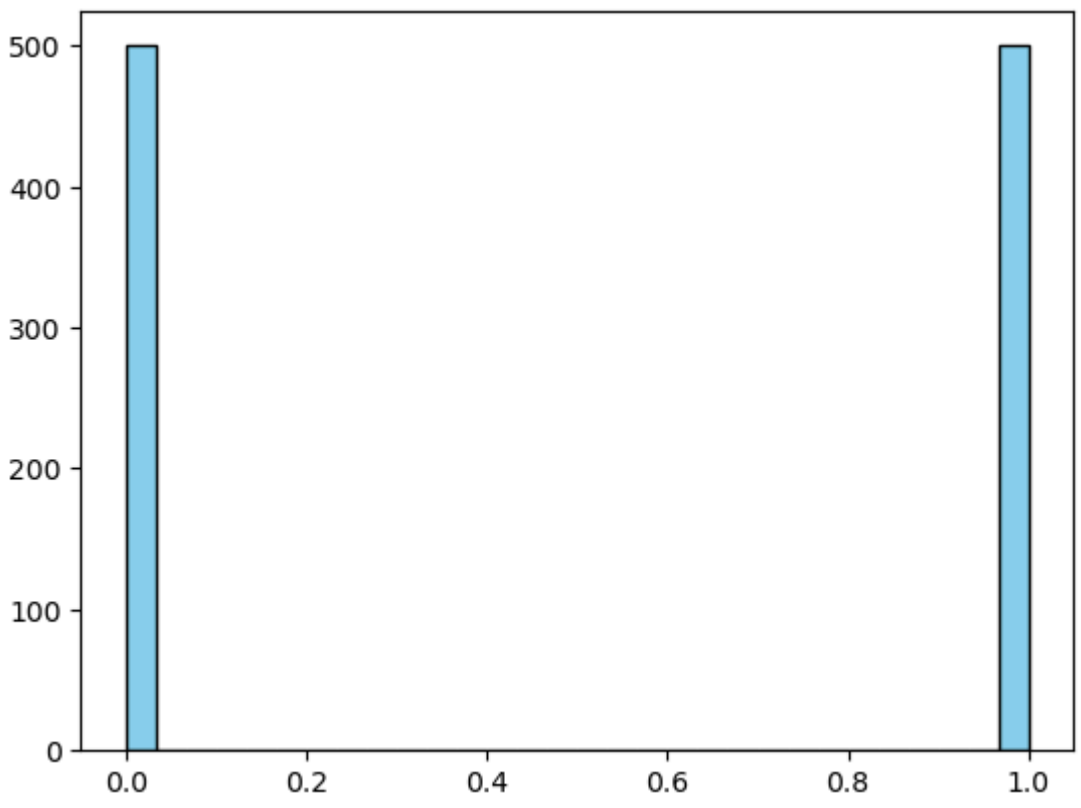


In [6]:
```
logreg = L2LogisticRegression(C=0)
logreg.fit(X, y)
y_proba = logreg.predict_proba(X)

print(f"coefs: {logreg.coefs}")
print(f"intercept: {logreg.intercept}")
print(f"Log Loss: {compute_log_loss(y, y_proba)}")
print(f"AUC: {compute_auc(y, y_proba)}")
print(f"ECE: {compute_ece(y, y_proba)}")
fig = plt.hist(y_proba, bins=30, color='skyblue', edgecolor='black')
```

```
coefs: [3.98825699 3.40827279]
intercept: [-35.23894751]
Log Loss: 8.908908788551315e-06
AUC: 1.0
ECE: 8.901337452000311e-06
```

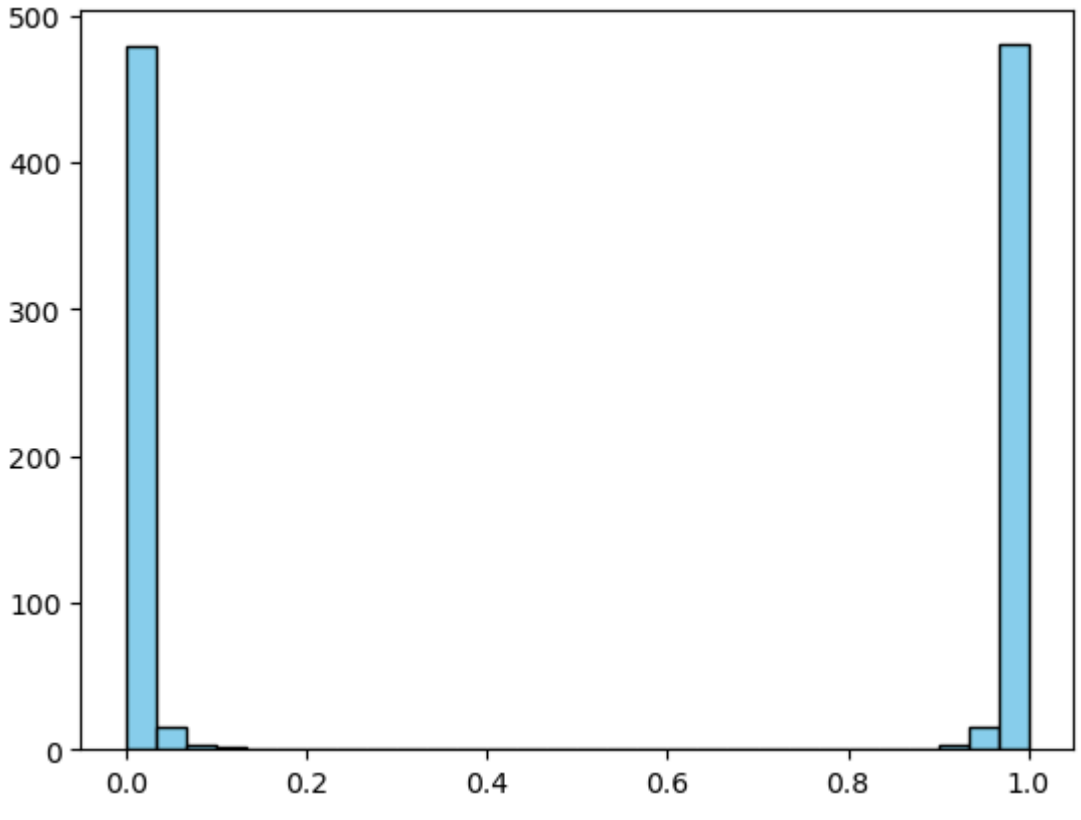
```
c:\Users\issac\miniforge3\envs\dsc80\Lib\site-packages\cvxpy\problems\problem.py:1539: UserWarning: Solution may be inaccurate.
Try another solver, adjusting the solver settings, or solve with verbose=True for more information.
  warnings.warn(
```



In [7]:
```
logreg = L2LogisticRegression(C=0.01)
logreg.fit(X, y)
y_proba = logreg.predict_proba(X)

print(f"coefs: {logreg.coefs}")
print(f"intercept: {logreg.intercept}")
print(f"Log Loss: {compute_log_loss(y, y_proba)}")
print(f"AUC: {compute_auc(y, y_proba)}")
print(f"ECE: {compute_ece(y, y_proba)}")
fig = plt.hist(y_proba, bins=30, color='skyblue', edgecolor='black')
```

```
coefs: [0.90541966 0.93475659]
intercept: [-9.14607445]
Log Loss: 0.00868488516380167
AUC: 1.0
ECE: 0.008520243340574449
```



Part (c)

Generate a dataset of $n = 1000$ points by calling the function

```
X, y = generate_data_4c(n = 1000)
```

This function generates $n = 1000$ examples from the following distribution:

$$\begin{aligned} x_1, x_2 &\sim \textsf{Normal}([5, 5], I) \\ p &= \text{logistic}(10 - x_1 - x_2) \\ y &\sim \textsf{Bernoulli}(p) \end{aligned}$$

i) Plot the positive and negative samples in your dataset. Do you expect this classification task to be easier or harder in comparison from part (b)? Why or why not?

ii) What are the coefficients that you should expect if you were to fit a logistic regression model on a very large number of samples on this dataset?

iii) Fit a vanilla logistic regression model. How do the coefficients of the vanilla logistic regression model you fit compare to these value you stated in the previous part?

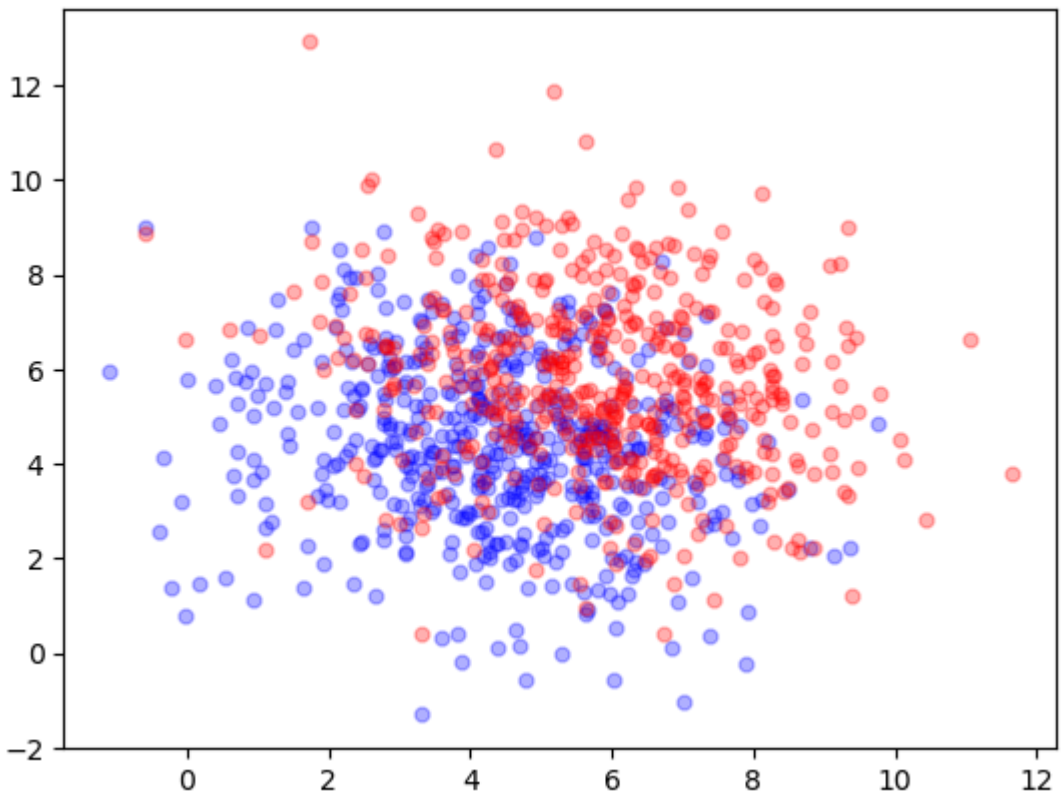
iv) What is the value of your training loss, AUC, and ECE of your model. Explain how you expect each of these summary statistics to change as you fit a model using a very large number of samples on this dataset?

v) Would you expect to improve prediction accuracy on this dataset by fitting a penalized logistic regression model and tuning the ℓ_2 penalty? Why or why not?

vi) Would you expect to improve prediction accuracy on this dataset by fitting a model with basis functions for x_1 and x_2? Why or why not

In [8]:
```
def generate_dataset_4c(n = 100, std = 2.0,  seed = 1):
    assert n > 2
    n = int(n)
    X, _ = datasets.make_blobs(n_samples=n, centers = [(5, 5)], cluster_std = std,  n_features = 2, random_state = seed)
    scores = 10.0 - X[:, 0] - X[:, 1]
    p = inverse_logit(scores / std)
    y = np.random.binomial(1, p)
    return X, y
```

In [9]:
```
X, y = generate_dataset_4c(n=1000)
plot_data(X, y)
```



In [10]:
```
logreg = L2LogisticRegression(C=0)
logreg.fit(X, y)
y_proba = logreg.predict_proba(X)

print(f"coefs: {logreg.coefs}")
print(f"intercept: {logreg.intercept}")
print(f"Log Loss: {compute_log_loss(y, y_proba)}")
print(f"AUC: {compute_auc(y, y_proba)}")
print(f"ECE: {compute_ece(y, y_proba)}")
fig = plt.hist(y_proba, bins=30, color='skyblue', edgecolor='black')
```

```
coefs: [-0.52707167 -0.51052774]
intercept: [5.30227402]
Log Loss: 0.53521735638364
AUC: 0.8060579873655236
ECE: 0.0184566705748682
```

